DOKUZ EYLUL UNIVERSTY

ENGINEERING FACULTY

DEPARTMENT OF COMPUTER ENGINEERING

CME 2210 OBJECT ORIENTED ANALYSIS AND DESIGN

FACTORY MANAGEMENT SYSTEM

FINAL REPORT

by

Atakan Kahraman 2022510066

Emre Altundal 2022510092

Serhat Aydın 2023510130

Lecturer

Prof. Dr. Semih Utku

11.06.2025

IZMIR

Contents

1. Phase 1: Analysis and Requirements Definition

- 1.1 System Definition and Objectives
- 1.2 Functional Requirements
- 1.3 Non-Functional Requirements
- 1.4 Use Cases
- 1.5 Use Case Diagram

2. Phase 2: System Design

- 2.1 System Architecture
- 2.2 Class Diagram
- 2.3 Sequence Diagram
- 2.4 State Chart Diagram
- 2.5 Activity Diagram
- 2.6 Overall System Design Description

3.1 Phase 3: Implementation

- 3.1.1 Implementation Details
- 3.1.2 System Coding Overview
- 3.1.3 Key Classes and Their Responsibilities
- 3.1.4 Data Handling and File Access

3.2 Code Structure and Class Design

- 3.2.2 Class Organization
- 3.2.3 How Classes Work Together
- 3.2.4 Example Flow: Creating a Customer Order

3.3 Testing Approach and Results

- 3.3.2 Testing Approach
- 3.3.3 Results and Observations

3.4 Reflection and Future Improvements

- 3.4.2 Reflections
- 3.4.3 Future Improvements

1. Phase 1

1.1 System Definition and Objectives

Factory management system is a software application which integrates manual labour duties, management responsibilities and optimizes them via automation.

The system will basically improve real-time observation, production tracking, resource distribution and workforce management while aiming to improve efficiency, reduce manual workload, boost overall productivity throughout the factory and assist decision-making.

This system will be built according to OOP principles which will provide modularity, scalability and maintainability.

1.2 Functional Requirements

Functionalities are described based on user types.

Logging in as employer or manager:

A manager in the Factory Management System is responsible for overseeing factory operations, ensuring efficiency, and making strategic decisions. The system provides managers with various tools to monitor production, manage employees, and optimize resources.

- Employee rating and performance review
- Production management
- Shift management
- Stock management
- Order management
- Financial monitoring

Logging in as employee:

An employee in the Factory Management System uses the system to track their work-related information, manage schedules, and stay updated on payroll, performance, and leave balances.

- Payroll monitoring
- Managing leave rights
- Display work rating
- Display shift information

Logging as a customer:

A customer in the Factory Management System utilizes the platform to place and manage orders, track their status, and access product information. The system makes the customer experience smooth and transparent by offering essential tools for managing their purchases and providing feedback.

- Placing and managing orders
- Reviewing order status
- Feedback system
- Reviewing stock information about certain products

1.3 Non-Functional Requirements

1. Performance

Response Time: The system should respond to user requests within 1-2 seconds under normal conditions.

Load Handling: The system must be able to scale and handle up to 10,000 records in real-time for inventory, orders, and employee data.

2. Scalability

Horizontal Scaling: The system should be able to scale horizontally to avoid facing problems in case of increasing number of users and data.

3. Security

Role-Based Access Control: The system must ensure that users can only access data relevant to their roles.

4. Maintainability

Modularity: The system should be modular and organized update easily, bug fixes, and new feature integrations without breaking the whole system.

Code Quality: The system should follow high quality codes and be properly documented to apply easy maintenance.

1.4 Use-Cases

Use Case 1: Log In to View Payroll and Shift Information

Participating Actor: Employee

Entry conditions:

- The employee has an active account in the system.
- The employee knows their login credentials (username and password).

Main Flow:

1. The employee navigates to the system's login page.
2. The employee enters their username and password.
3. The system authenticates the employee.
4. The system displays a menu with options to view payroll, insurance information, and shift schedule. The employee returns to menu.
5. The employee chooses Payroll, where they can view the salary. The employee returns to menu.
6. The employee chooses Shift Information, where they can view their assigned shift. The employee returns to menu.
7. The employee logs out after reviewing the information.

Exit conditions:

- The employee has viewed their payroll and shift details.

Use Case 2: Review Employee Performance and Approve Leave Request

Participating Actor: Manager

Entry conditions:

- The manager has logged in with the necessary role-based access.
- The employee has submitted a leave request through the system.

Main Flow:

1. The manager logs into the system using their password and username.
2. The manager gets a notification which indicates that there are leave requests on hold.
3. The manager moves to the Employee Performance section and selects an employee.
4. The system displays the employee's performance report.
5. The manager reviews the employee's work rating. The manager moves back to the menu.
6. The manager navigates to the Leave Requests section and reviews the pending leave request from the employee.
7. The manager approves, rejects or holds the leave request based on the employee's performance.
8. The system updates the employee's leave status.

Exit conditions:

- The employee's leave request is processed and updated in the system.
- The employee's performance review is recorded and can be seen by the employee and manager.

Use Case 3: Place and Track an Order

Participating Actor: Customer

Entry conditions:

- The customer has an active account.
- The customer is logged in.

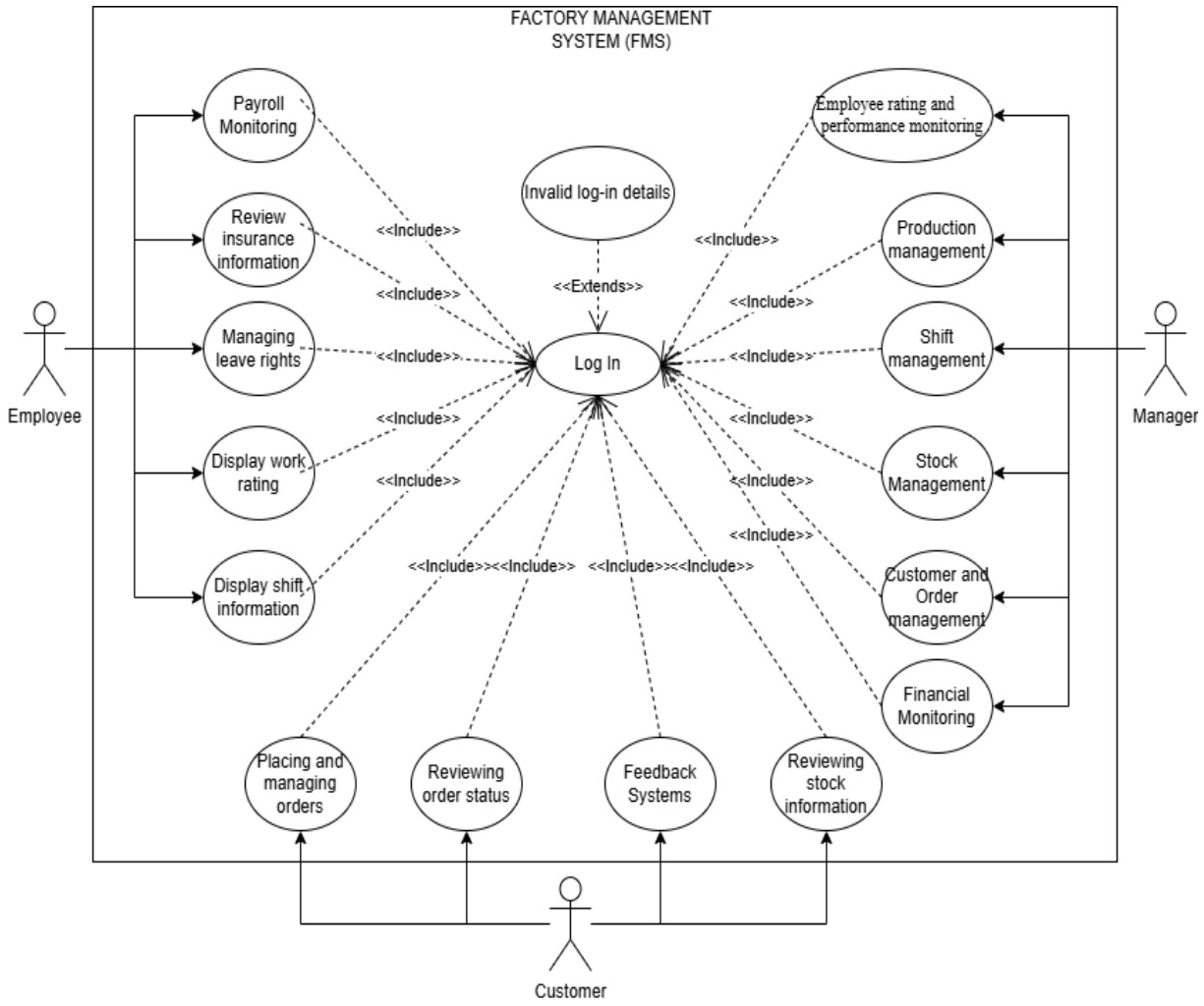
Main Flow:

1. The customer logs into the system.
2. The customer searches for available products and selects an item to purchase.
3. The system confirms the order and generates an order ID.
4. The customer navigates to Order Status to track the current status of their order.

Exit conditions:

- The customer's order is placed, and they can track its status in real-time.

1.5 Use Case Diagram



2. Phase 2

2.1 System Architecture

The Factory Management System is designed with a modular, scalable, and maintainable structure based on object-oriented programming principles. The system consists of three main layers: Presentation Layer, Application Layer, and Data Layer. Each of these layers contains specific modules that interact with each other through pre-defined rules.

1. **Presentation Layer**

   Displays different interfaces on the user type.

   Manager: Shift management, performance review, stock control, and financial operations.

   Employee: Viewing salary information, insurance status, work shifts, and performance scores.

   Customer: Placing orders, tracking order status, accessing product info, and providing feedback.

2. **Application Layer**

   Authentication Module: Handles login, verifies the user's role, and grants access accordingly.

   Employee Management Module: Manages employee data, leave requests, and performance tracking.

   Production and Shift Module: Tracks production processes and plans employee shifts.

   Order Management Module: Allows customers to place orders, checks stock, and updates order status.

   Finance and Payroll Module: Manages salary calculations, payment records, and insurance data.

3. **Data Layer**

   A txt/csv based database will be used.

   Stores structured data such as users, products, orders, salaries, shifts, and performance records.

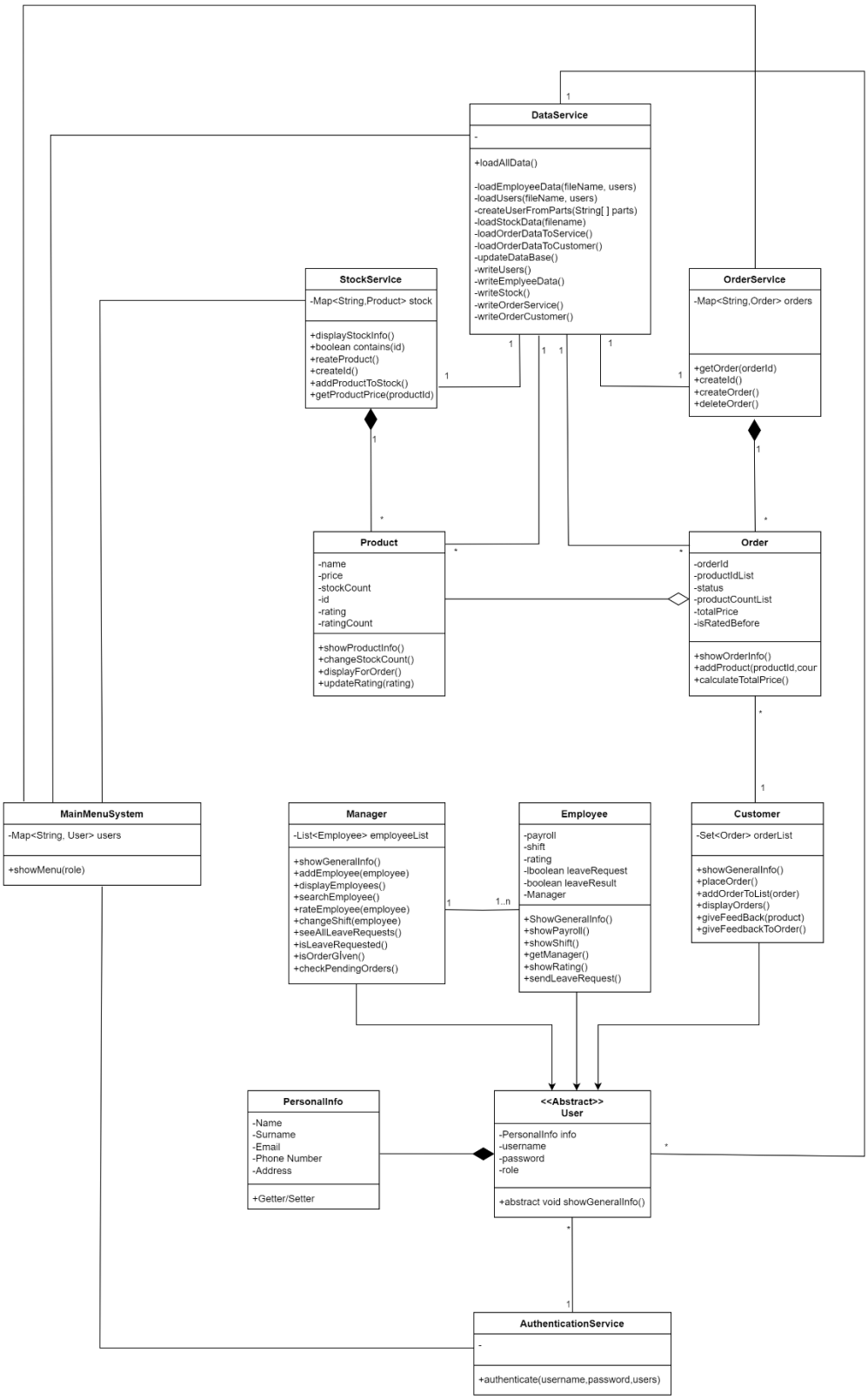
Layer Communication

When a user performs an action, a request is sent to the relevant module and a response is returned. (Example: An employee logs in → the system authenticates the user → they are directed to role-specific pages.)

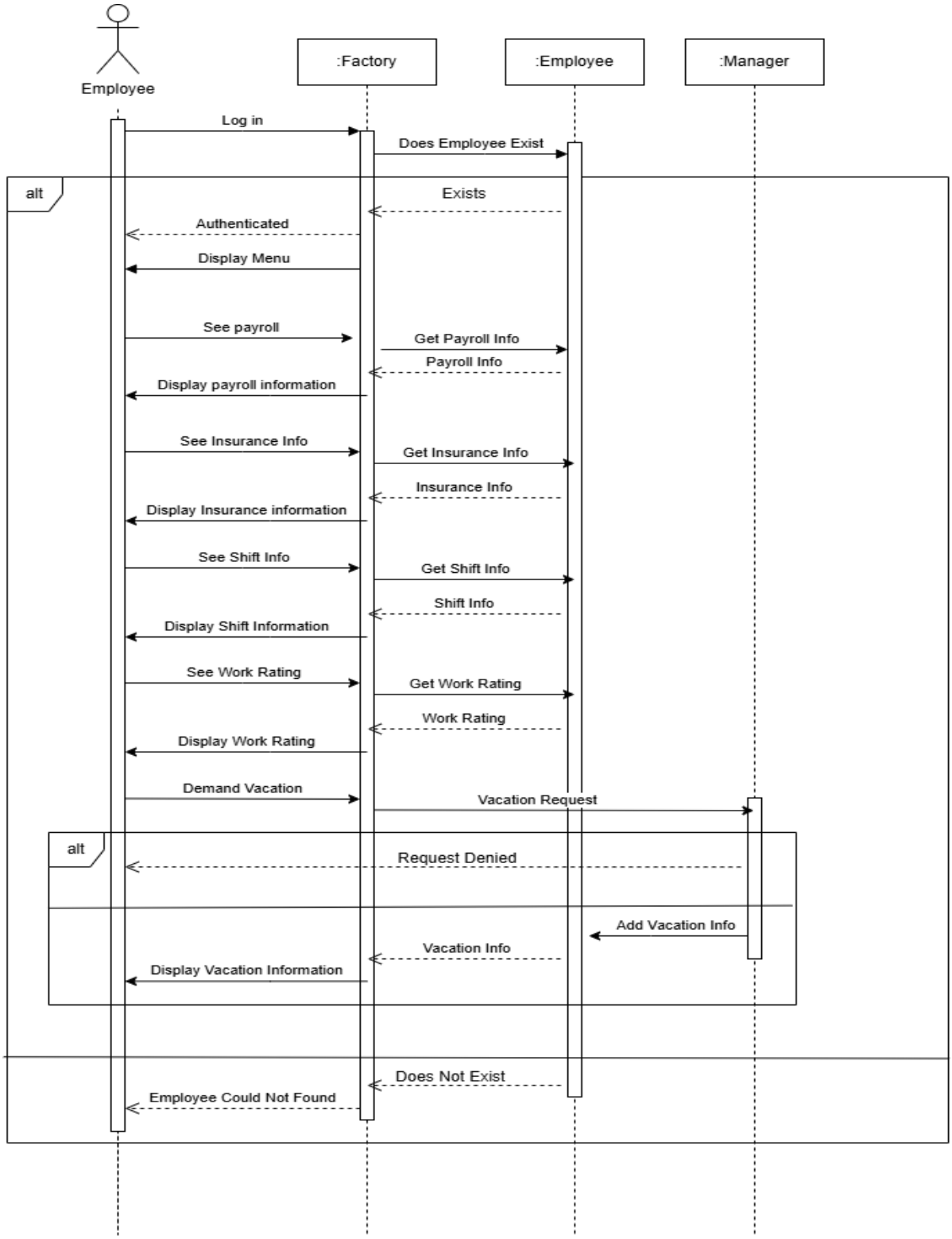
Security

Role-Based Access Control is implemented. Users can only access data and functions relevant to their roles. Also private data is stored securely using abstraction methods such as encapsulation.

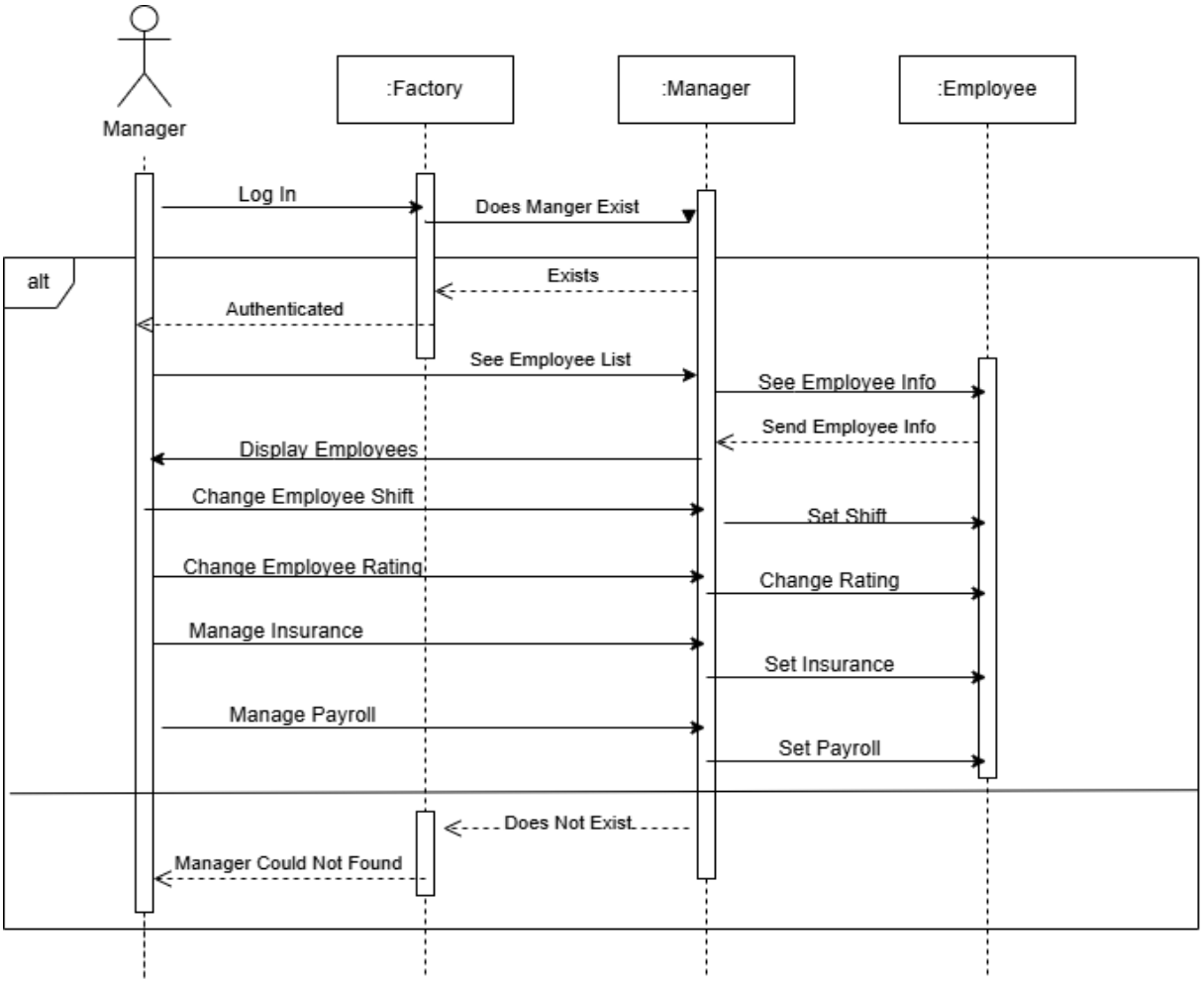
2.2 Class Diagram

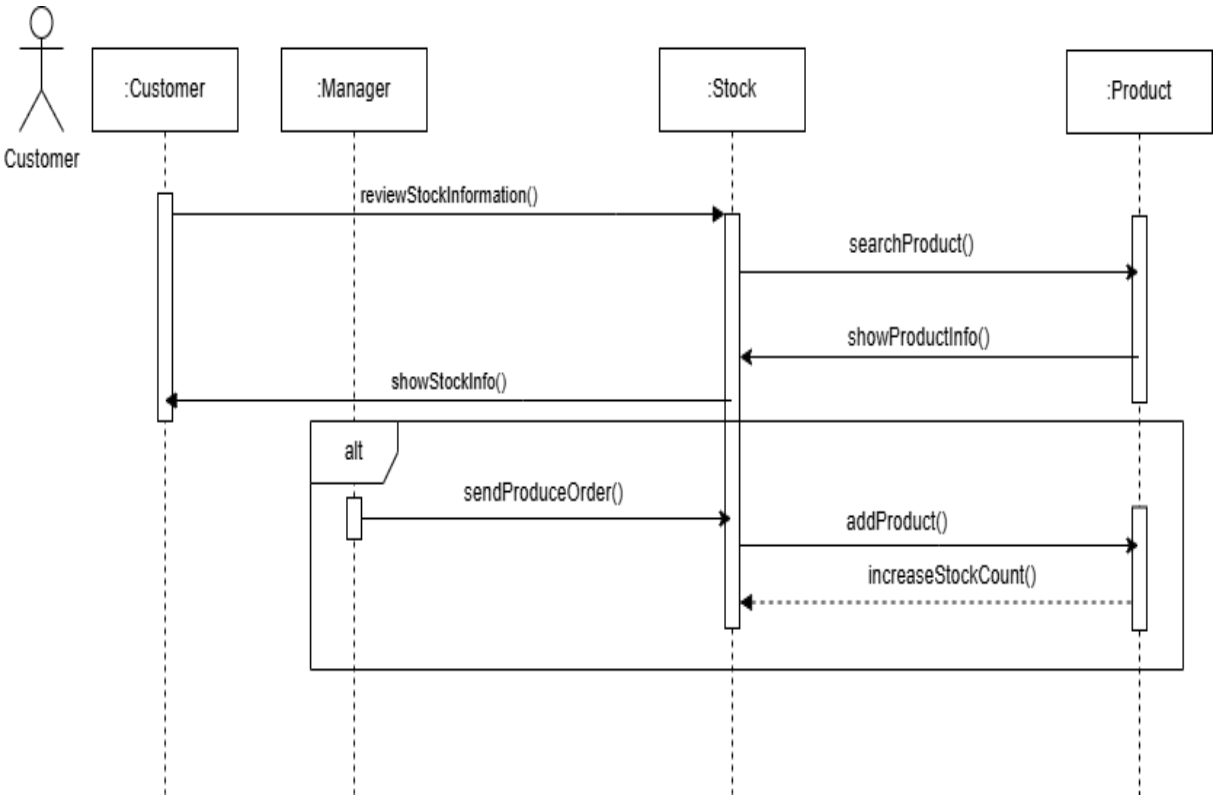


2.3.1 Sequence Diagram 1

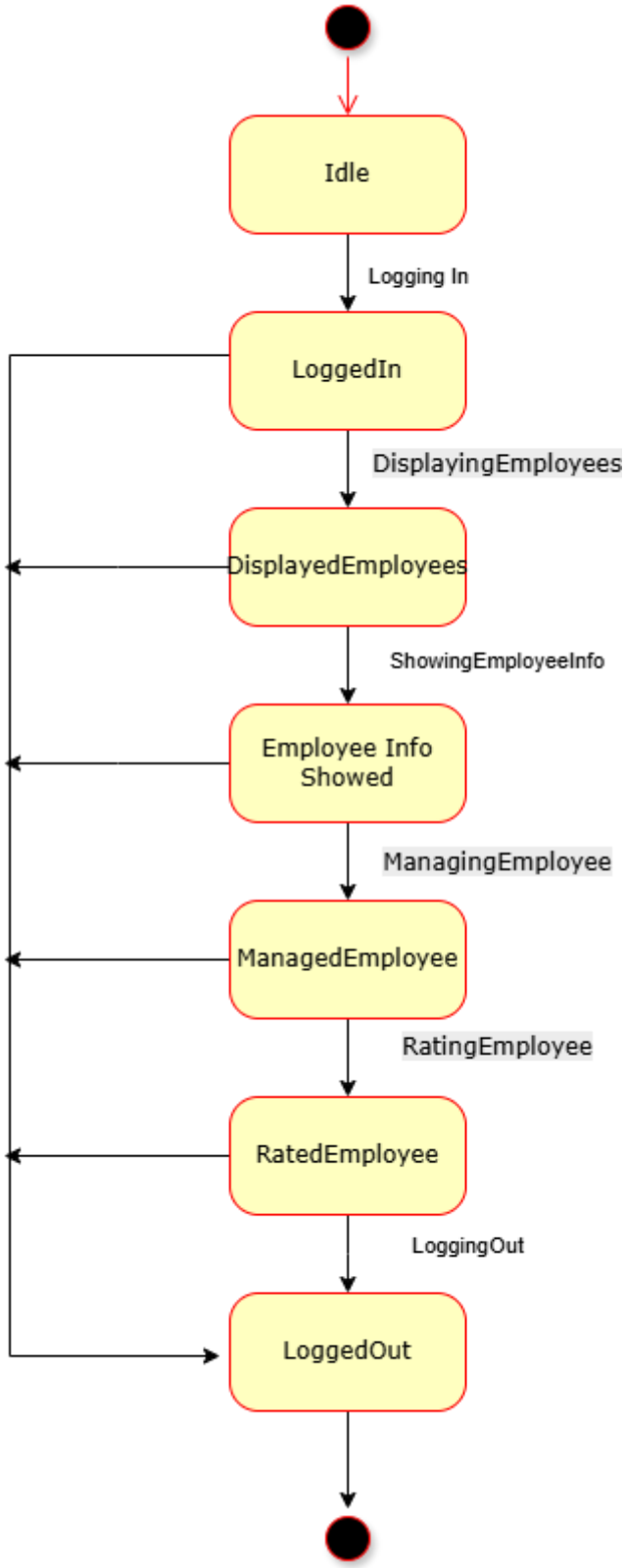


2.3.2 Sequence Diagram 2-3

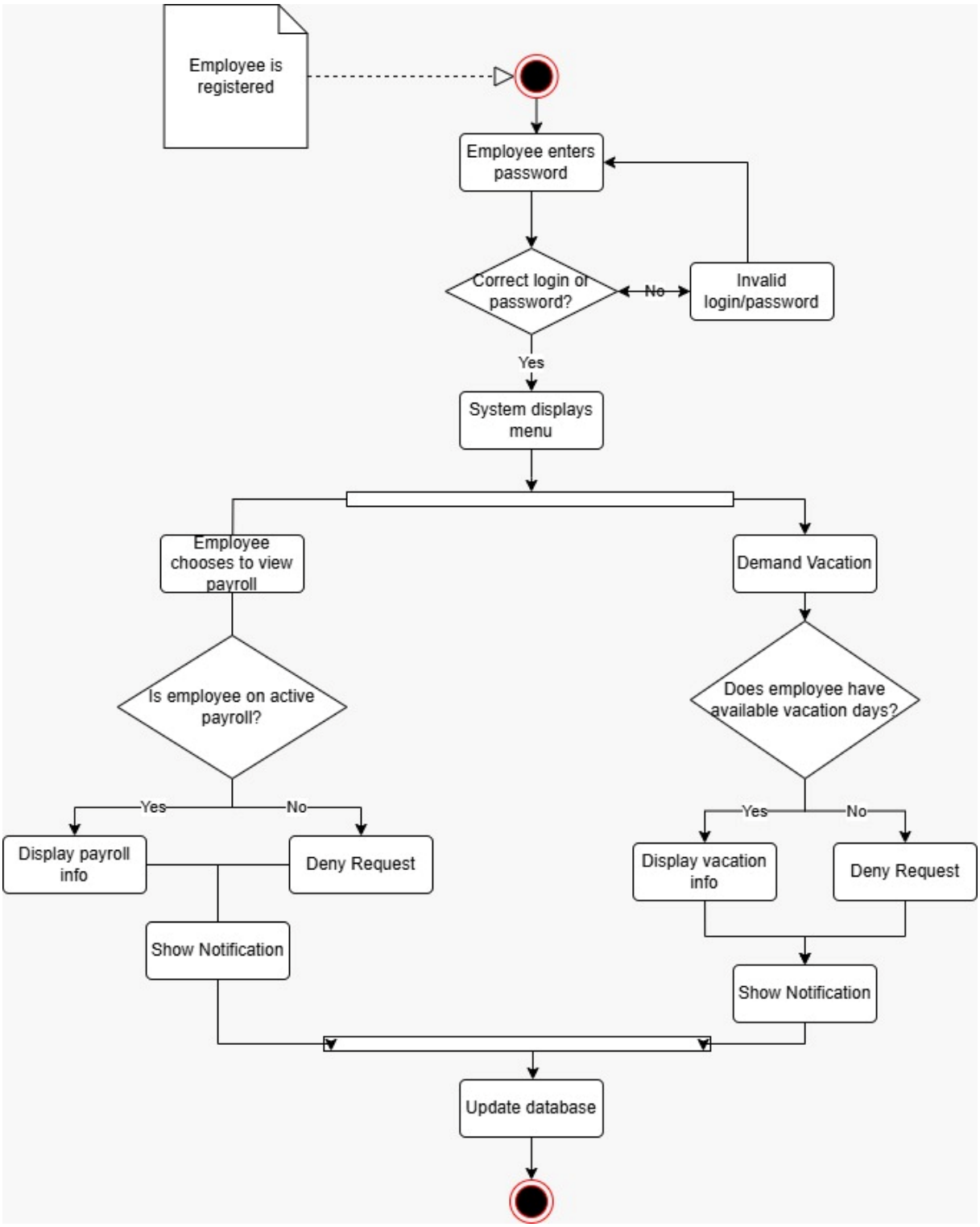




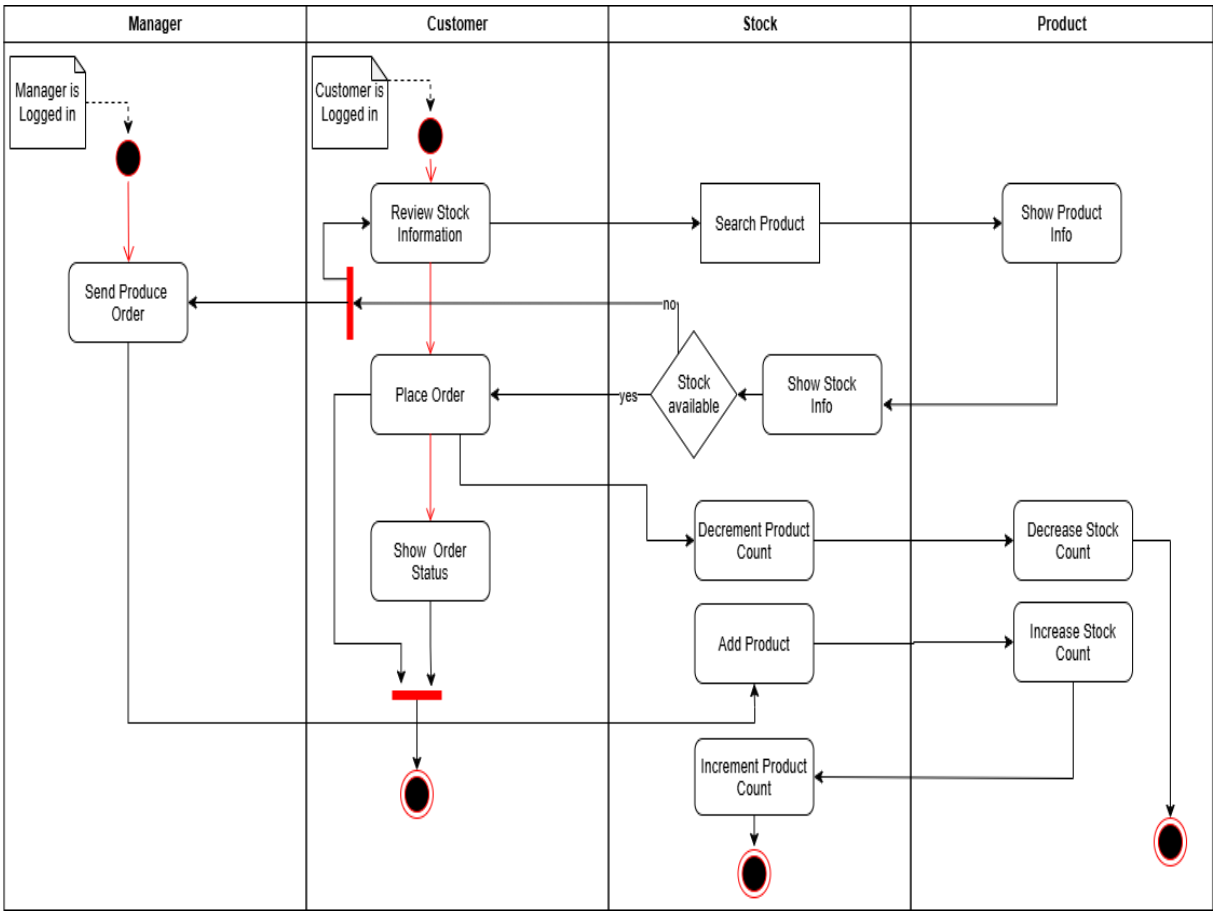
2.4 Statechart Diagram



2.5.1 Activity Diagram 1



2.5.2 Activity Diagram 2



2.6 Overall System Design Description

System Overview:

The Factory Management System is a role-based enterprise software to automate the process of factory operations. The system provides functionalities such as production monitoring, employee performance evolution, shift scheduling, stock controlling and customer order tracking. The main object is to improve effectiveness of the factory and reduce the manual effort.

User Roles and Functionalities:

Manager

- Monitoring production and employee performance
- See, approve, reject leave request
- Manage stocks, production, employee shifts
- Check payrolls

Employee

- View shift schedule and payroll
- Demand vacation
- See personal performance ratings

Customer

- Place and manage product orders
- Submit feedback on product experience
- See order status and review product availability

Design Principles

The system follows Object-Oriented Programming principles such as encapsulation, inheritance. This allows reusability of components, and easier debugging.

3.1 Phase 3

3.1.1 Implementation Details

The developed system has been implemented in Java, following the object-oriented principles defined in the earlier design phases. The Java implementation is structured into multiple interrelated classes, each with specific responsibilities to ensure modularity, maintainability, and scalability.

3.1.2 System Coding Overview

The system is divided into the following core components:

- AuthenticationService: Handles user authentication by verifying credentials from the user database.
- MainMenuSystem: Acts as the central interface controller, directing users to various modules based on their roles (e.g., customer, employee, manager).
- DataService: Responsible for reading and writing data from CSV files such as user information, product details, order history, and stock data.
- OrderService: Manages the creation and processing of customer orders, including adding items, calculating totals, and storing order records.
- StockService: Monitors and updates the inventory levels based on incoming orders and restocking operations.

3.1.3 Key Classes and Their Responsibilities

- User.java: Serves as an abstract base class for user types. It holds shared attributes such as username, password, and role.
- Customer.java / Employee.java / Manager.java: These subclasses extend User and implement role-specific features. For instance, Customer can place orders, while Manager can review reports.
- Product.java: Represents each product in the inventory, including product ID, name, stock level, and price.
- Order.java: Encapsulates an individual customer order, storing order items and associated customer data.
- PersonalInfo.java: Contains personal data related to users, improving data encapsulation and separation of possible problems.

Each class supports encapsulation, and polymorphism is used for user role behavior handling. Composition is used for aggregating user information and order data, ensuring a loosely coupled design.

The implementation shows a layered architecture approach, separating concerns such as user interaction, business logic, and data access. This enables better modularity and makes the system easier to test and maintain. Each functional unit is clearly encapsulated in its respective class, minimizing dependencies and supporting reusability.

Also, the system uses standard Java libraries for file handling, exception management, and collection processing. This ensures compatibility and reliability. For instance, CSV file handling is accomplished using buffered readers and parsers to ensure efficient I/O operations with large datasets.

The program also includes a clean user experience through a menu-based console interface, which dynamically changes based on the authenticated user's role. This dynamic control is implemented using polymorphic behavior and runtime dispatch, allowing role-specific functionality without duplicating logic.

In summary, the Java implementation is mainly accurate when compared to the original system design, with respect to class responsibilities, module separation, and extensibility. This program base allows us to adapt easily to new requirements or system extensions in the future.

3.1.4 Data Handling and File Access

The system does not use a traditional SQL database but instead reads and writes data from CSV files, such as customer_orders.csv, employee_data.csv, and stock_data.csv. This decision keeps the system lightweight and file-based, making it suitable for smaller projects.

The DataService class is responsible for handling file input and output operations. It uses Java's BufferedReader and FileReader classes to load data line-by-line from CSV files and parse them into appropriate objects like Customer, Order, and Product. When changes are made, such as adding a new order, the system updates the relevant file accordingly using FileWriter and BufferedWriter.

This approach also simplifies testing and portability since there is no need for external database setup or configuration.

3.2.1 Code Structure and Class Design

The code is organized into separate classes, each doing specific jobs. This makes the system easier to understand, test, and change in the future. The project follows object-oriented programming principles like encapsulation, inheritance, and polymorphism.

3.2.2 Class Organization

Each Java class in the system has a clear role. Here's how they are grouped and what they do:

- User.java: This is the main class for users. It stores shared info like username and password.
- Customer.java, Employee.java, Manager.java: These classes extend User. Each has extra features depending on the role. For example, Customer can place orders and Manager can view reports.
- Product.java: Holds data about each product, like name, ID, price, and quantity in stock.
- Order.java: Keeps all order details, such as which customer made the order and which products were bought.
- PersonalInfo.java: Stores extra details about users, such as name or contact info. It is used to keep things separate and tidy.
- AuthenticationService.java: Checks login details to see if a user is allowed to enter the system.
- MainMenuSystem.java: Shows different options to users depending on their role.
- DataService.java: Reads and writes user, product, and order data from CSV files.
- OrderService.java: Manages new orders, adds items to orders, and stores them.
- StockService.java: Tracks stock levels and updates inventory when needed.

3.2.3 Class Interaction

The classes interact through method calls and data sharing. For example, OrderService uses Product and Customer to create and save an order. MainMenuSystem uses AuthenticationService to check login and show the right menu.

The system is designed so that parts can be updated without changing the whole program. This makes it flexible and easy to improve in the future. All classes use simple, clear methods to do their work, making the system clean and easy to understand.

3.2.4 How Classes Work Together

The classes are designed to interact in a clear and organized way, similar to what a simple UML diagram would show. For example:

- MainMenuSystem uses AuthenticationService to check login.
- Once logged in, it calls the correct services like OrderService or StockService based on the user role.
- OrderService needs information from Product and Customer to create a new order.
- DataService is used by many classes to read and write data from CSV files.

Example Flow: Creating a Customer Order

Here is how the system works when a customer places an order:

1. The customer logs in. MainMenuSystem calls AuthenticationService to check username and password.
2. After login, MainMenuSystem shows the customer menu.
3. The customer chooses to create a new order. MainMenuSystem calls OrderService.
4. OrderService loads available products using Product and DataService.
5. The customer selects products. OrderService creates an Order object with that data.
6. OrderService saves the new order using DataService, which writes it into the order_contents.csv file.
7. StockService is called to update the product quantity.

This flow shows how classes are used together to complete a real task, and how responsibilities are shared across them without overlap. Each class focuses on one job and uses other classes only when needed.

3.3.1 Testing Approach and Results

To ensure the reliability and correctness of the system, several testing methods were used during development. Testing was focused on both individual class behavior (unit testing) and overall system interaction (integration testing).

3.3.2 Testing Approach

1. Unit Testing: Each major class such as OrderService, DataService, and AuthenticationService was tested with different input cases to verify that methods returned correct results. For example, login tests included valid, invalid, and empty credentials.

2. Integration Testing: The interaction between modules was tested by simulating real user flows, such as logging in, placing an order, and updating stock. This ensured that data flowed correctly between classes and that shared resources like CSV files were properly updated.

3. Boundary and Error Testing: Edge cases like empty files, incorrect user roles, or stock levels below zero were tested to verify that the system responded with proper error handling and did not crash unexpectedly.

3.3.3 Results and Observations

- The authentication system correctly handled valid and invalid login attempts.
- The order creation process worked as expected, linking customers with products and correctly updating the inventory.
- CSV file operations were stable across different test cases, and data integrity was preserved after each write operation.
- In boundary tests, the system informed users of errors, such as trying to place an order with insufficient stock.
- The modular structure allowed easy isolation and testing of specific functionality without affecting other parts of the code.

Overall, the system performed reliably in all tested scenarios. The code structure made it easy to identify bugs and apply fixes, and the results confirmed that the application is good enough for practical usage.

3.4.1 Reflection and Future Improvements

Developing this system provided valuable experience in designing and implementing object-oriented applications. It also showed the importance of clear class responsibilities, modular design, and effective data handling practices using CSV files.

3.4.2 Reflections

One of the strengths of this project was its modular structure. Breaking the system into small, focused classes made the code easier to understand and maintain. Using a console-based menu also helped test each module without requiring a graphical interface.

Another key takeaway was the benefit of separating features. For example, keeping data access methods in a separate DataService class helped avoid duplicated logic and made file operations easier to test and debug.

The testing process helped identify several edge cases early, and catching these problems during development made the system more stable. These practices emphasized the value of continuous testing throughout the development lifecycle.

3.4.3 Future Improvements

While the system meets its current goals, there are several areas for potential improvement:

- **Switch to a relational database:** Using a real database system like SQLite or MySQL would make data storage more scalable and secure.
- **GUI Implementation:** Replacing the console interface with a graphical user interface would improve the user experience and make the system more accessible to non-technical users.
- **Advanced Role Permissions:** Adding more detailed role-based access control could help better manage user privileges.
- **Data Validation and Logging:** More advanced input validation and logging mechanisms would enhance system reliability and traceability.

Overall, this project forms a strong base that can be expanded into a more advanced and user-friendly application with future improvements.